DATABASE DESIGN

DataCamp Associate Data Engineer

MASTER STUDY NOTES

Part I — Module-ordered notes
Exact topic order from the supplied finaledit.pdf,
with missing StackEdit images restored from the supplied image assets.

Part II — Expanded companion notes
The previously created consolidated guide with explanations,
comparisons, SQL patterns, exam rules and additional visuals.

All diagrams/images are embedded directly in this PDF.

Database Design — Associate Data Engineer in SQL

> **Exam-focused master notes** compiled from the provided `notes.md`, DataCamp Database Design chapter PDFs, and the supplied quiz/screenshots.
>
> The structure is intentionally Markdown-first so it can be pasted into **StackEdit, Dillinger, VS Code, Obsidian, or GitHub**.

---

0. What this module covers

The Database Design course focuses on how data should be processed, stored, organized, secured, and queried efficiently. The current DataCamp course structure includes four major areas: processing/storing/organizing data, schemas & normalization, views, and database management. DataCamp Database Design course overview

The mental model

```
Business requirement
        ↓
Choose processing pattern (OLTP / OLAP)
        ↓
Choose storage (DB / warehouse / lake)
        ↓
Choose data model + schema
        ↓
Normalize or denormalize deliberately
        ↓
Add keys / constraints
        ↓
Create useful views
        ↓
Control access with roles
        ↓
Partition / integrate data when needed
        ↓
Choose the appropriate DBMS
```

---

1. Processing, storing, and organizing data

1.1 OLTP vs OLAP

OLTP — Online Transaction Processing

Designed for **day-to-day operational transactions**.

Typical tasks:

- Find the price of a book.
- Update the latest customer transaction.
- Keep track of employee hours.
- Frequent inserts/updates.
- Small, simple, fast transactions.

OLAP — Online Analytical Processing

Designed for **analysis, reporting, and decision-making**.

Typical tasks:

- Calculate books with the best profit margin.
- Find the most loyal customers.
- Decide employee of the month.
- Complex aggregations and analytical queries.
- Historical/consolidated data.

OLTP vs OLAP — memorize this table

Feature	OLTP	OLAP
Purpose	Support daily transactions	Report & analyze data
Design	Application-oriented	Subject-oriented
Data	Up-to-date, operational	Consolidated, historical
Typical size	Snapshot / GB-scale	Archive / TB-scale
Queries	Simple, frequent	Complex, aggregate
Updates	Frequent	Limited
Users	Thousands	Hundreds
Typical workload	Write-intensive	Read-intensive

Exam shortcut:

> **OLTP = operational + writes + current data.**
>
> **OLAP = analytical + reads + historical data.**

Source slides: chapter 1, pp. 4–8.

1.2 Structured, semi-structured, and unstructured data

Structured data

- Follows a schema.
- Defined data types and relationships.
- Example: SQL tables in a relational database.

Unstructured data

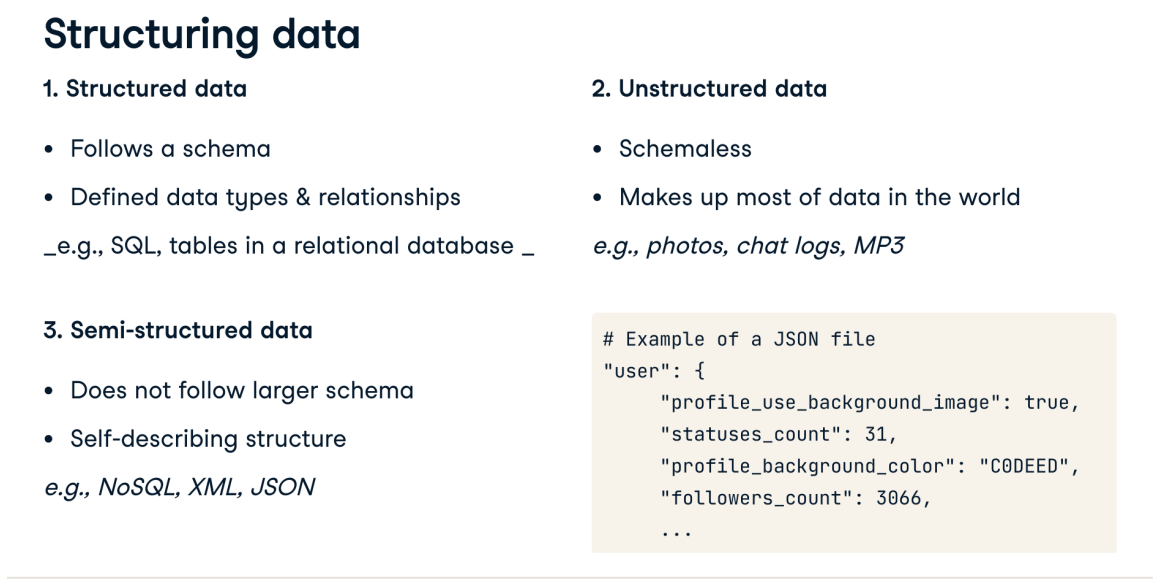
- Schemaless.

- Includes a large share of real-world data.
- Examples: photos, chat logs, MP3 files.

Semi-structured data

- Does not follow one large rigid schema.
- Has a self-describing structure.
- Examples: JSON, XML, many NoSQL systems.

Source: chapter 1, p. 11.



1.3 Traditional databases vs data warehouses vs data lakes

Traditional database

- Real-time relational structured data.
- Commonly associated with **OLTP**.

Data warehouse

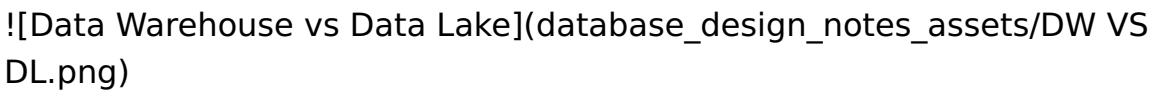
- Analyzes archived structured data.
- Optimized for **OLAP**.
- Organized for reading and aggregation.
- Usually read-only or read-mostly.
- Can contain data from multiple sources.

- Often uses **MPP — Massively Parallel Processing**.
- Commonly uses denormalized schemas and dimensional modeling.
- A **data mart** is a subset of a warehouse focused on a specific topic.

Data lake

- Stores many/all data types at relatively low cost.
- Can contain raw data, operational data, IoT logs, real-time data, relational and non-relational data.
- Can retain very large volumes, including petabytes.
- Uses **schema-on-read**, unlike the traditional schema-on-write approach.
- Needs cataloging/governance; otherwise it can become a **data swamp**.
- Useful for big-data analytics, Spark/Hadoop workloads, deep learning, and data discovery.

Source: chapter 1, pp. 13–15.



Data lake vs data warehouse — exam pattern

Data Lake	Data Warehouse
Can include unstructured data	Contains structured data
Purpose may not yet be known	Purpose is known
Less organized	More organized
Flexible	More controlled
Schema-on-read	Schema-on-write
Good for exploration / big data	Good for analytics / reporting

---

1.4 ETL vs ELT

- **ETL:** Extract → Transform → Load.
- **ELT:** Extract → Load → Transform.

Remember the conceptual difference: **ETL transforms before loading; ELT loads before transforming.**

---

2. Database design and data modeling

2.1 What is database design?

Database design determines how data is logically stored and how it will be read and updated. It uses database models and schemas.

A **schema** is the blueprint of a database. It defines things such as:

- tables
- fields/columns
- relationships
- indexes
- views

In a relational database, inserted data must respect the schema.

Source: chapter 1, p. 19.

2.2 Three levels of data modeling

Conceptual data model

Describes:

- entities
- relationships
- attributes
- business requirements

Typical tools: ER diagrams, UML diagrams.

Logical data model

Defines:

- tables
- columns

- relationships

Typical tools: relational schemas and star schemas.

Physical data model

Describes how the model is physically stored/implemented.

Examples:

- partitions
- CPUs
- indexes
- backup systems
- tablespaces
- file/storage structures

Source: chapter 1, pp. 20–21.

Data modeling

Process of creating a *data model* for the data to be stored

1. Conceptual data model: describes entities, relationships, and attributes

- *Tools:* data structure diagrams, e.g., entity-relational diagrams and UML diagrams

2. Logical data model: defines tables, columns, relationships

- *Tools:* database models and schemas, e.g., relational model and star schema

3. Physical data model: describes physical storage

- *Tools*: partitions, CPUs, indexes, backup systems and tablespaces

[1] https://en.wikipedia.org/wiki/Data_model

Quiz mapping

Model	Think
Conceptual	Entities, attributes, relationships + business requirements
Logical	Tables, columns, relationships + relational model
Physical	Actual storage implementation / files / indexes / partitions

---

3. Dimensional modeling

Dimensional modeling is an adaptation of the relational model for data-warehouse design. It is optimized for OLAP/aggregation and commonly uses a **star schema**. It is designed to be easy to interpret and extend.

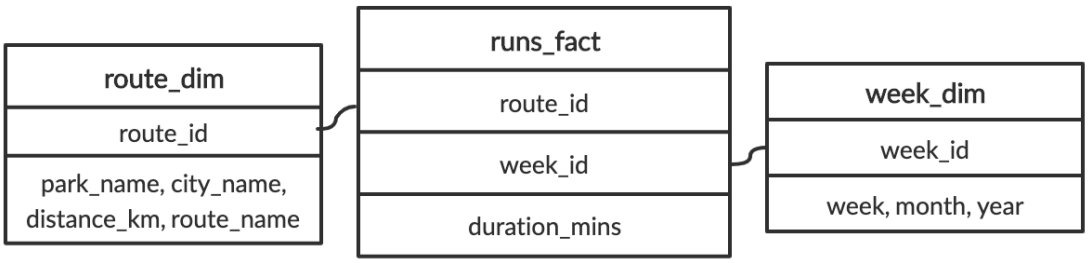
Fact tables

- Decided by the business use case.
- Hold records of metrics/measures.
- Change regularly.
- Connect to dimensions through foreign keys.

Dimension tables

- Hold descriptive attributes.
- Usually change less often.

Source: chapter 1, pp. 23–24.



---

4. Star schema vs snowflake schema

4.1 Star schema

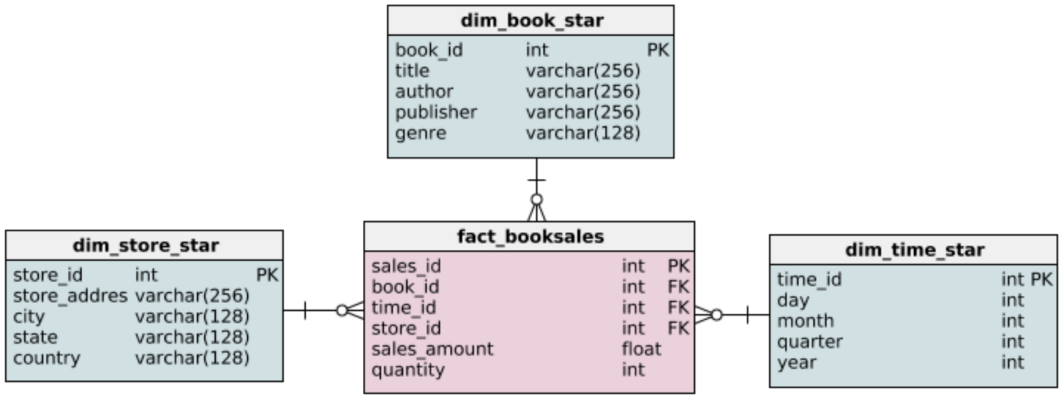
A star schema is the simplest dimensional model.

- One central **fact table**.
- Multiple **dimension tables** around it.

- Fact table stores metrics.
- Dimensions store descriptive information.
- Dimensions are generally denormalized.
- Usually fewer joins → simpler/faster analytical queries.

The DataCamp example uses book sales with dimensions for **book, time, and store**. The fact table contains measures such as sales amount and quantity. ✂cite turn0search2☎

Star schema example



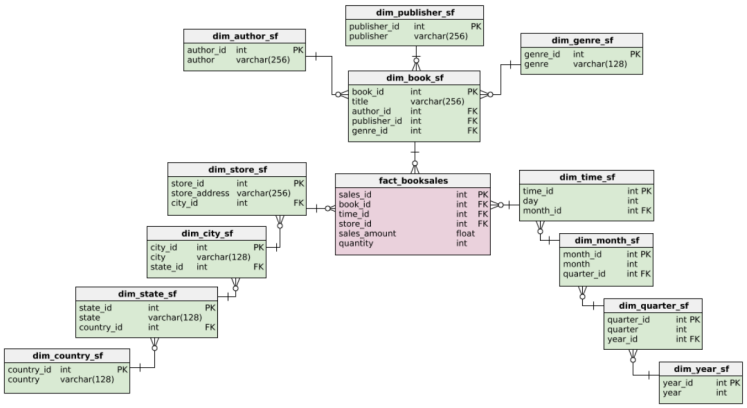
datacamp DATABASE DESIGN

4.2 Snowflake schema

A snowflake schema is an extension of a star schema.

- The **fact table can remain the same**.
- Dimension tables are split into additional related tables.
- Dimensions are more normalized.
- Example: `book → author / publisher / genre` and `store → city → state → country`.

Snowflake schema (an extension)



Star vs snowflake

Star	Snowflake
Denormalized dimensions	Normalized dimensions
Fewer tables	More tables
Fewer joins	More joins
Simpler analytical queries	More complex analytical queries
Easier to understand	More normalized / less redundant
Often faster for reads	Better consistency / extension in many cases

Source: chapter 2, pp. 2–6.

---

5. Normalization

5.1 Definition

Normalization is a database design technique that:

1. Divides tables into smaller tables.
2. Connects them using relationships.
3. Identifies repeating groups.
4. Reduces redundancy.
5. Improves data integrity.

Source: chapter 2, pp. 6–7.

5.2 Why normalize?

Benefits

- Less redundant data.
- Saves storage.
- Better data consistency.
- Safer updates, inserts, and deletes.
- Easier extension/redesign.

Costs

- More tables.
- More joins.
- More complex queries can require more CPU.

OLTP vs OLAP connection

OLTP: typically highly normalized and write-intensive; prioritize safe/fast data insertion.

OLAP: typically less normalized and read-intensive; prioritize fast analytical queries.

Source: chapter 2, pp. 20–24.

---

6. Normal forms — must know

Normal forms progress from less normalized to more normalized. The course focuses on **1NF, 2NF, and 3NF**.

6.1 1NF — First Normal Form

Rules:

- Every record/row must be unique.
- Every cell must contain **one value**.

- No repeating/multi-valued groups in a single cell.

Example

Bad:

```
student_id | email            | courses_completed
235        | jim@gmail.com    | Intro Python, Intermediate Python
```

1NF separates the multi-valued course information into rows/table structures so each cell contains one value.

Source: chapter 2, pp. 28–30.

Notes.md exercise

The car-rental example creates a separate `cust_rentals` table and removes multi-valued `cars_rented` and `invoice_id` columns from `customers`.

```sql
CREATE TABLE cust_rentals (
  customer_id INT NOT NULL,
  car_id VARCHAR(128) NULL,
  invoice_id VARCHAR(128) NULL
);

ALTER TABLE customers
DROP COLUMN cars_rented,
DROP COLUMN Invoice_id;
```

6.2 2NF — Second Normal Form

Must satisfy **1NF**, and:

- If the primary key is a single column, the table automatically satisfies 2NF.
- If the primary key is composite, every non-key attribute must depend on **the whole composite key**, not only part of it.

Memory phrase

> **2NF = no partial dependency.**

Source: chapter 2, pp. 31–32.

Notes.md car example

Move car attributes into a separate `cars` table and remove them from `customer_rentals`.

6.3 3NF — Third Normal Form

Must satisfy **2NF**, and:

- No transitive dependencies.
- A non-key column must not depend on another non-key column.

Memory phrase

> **3NF = no transitive dependency.**

Source: chapter 2, pp. 33–34.

Notes.md car example

Create `car_model(model, manufacturer, type_car)` and remove the dependent attributes from `rental_cars`.

---

7. Data anomalies

Insufficient normalization can cause three classic anomalies:

Update anomaly

Redundant copies of the same fact must all be updated. If one is missed, data becomes inconsistent.

Insertion anomaly

You cannot insert a fact because unrelated/missing attributes are required.

Deletion anomaly

Deleting one record unintentionally deletes the only copy of another fact.

Source: chapter 2, pp. 35–39.

Memory: `U-I-D` → **Update, Insertion, Deletion**.

---

8. Fact/dimension exercise — running example

Original `runs` table:

```
runs
- duration_mins FLOAT
- week INT
- month VARCHAR(160)
- year INT
- park_name VARCHAR(160)
- city_name VARCHAR(160)
- distance_km FLOAT
- route_name VARCHAR(160)
```

Best dimensional organization:

> Fact table holds `duration_mins`, `distance_km`, and foreign keys to dimensions containing route and week details.

The notes then create route/week dimensions and query `runs_fact` by joining `week_dim`.

Important correction: the supplied notes show `CREATE TABLE route(` for the week dimension. That is a likely exercise-note typo; conceptually the table should be named something like `week_dim` / `week`, not `route`.

---

9. Foreign keys in dimensional schemas

Foreign keys are essential because they:

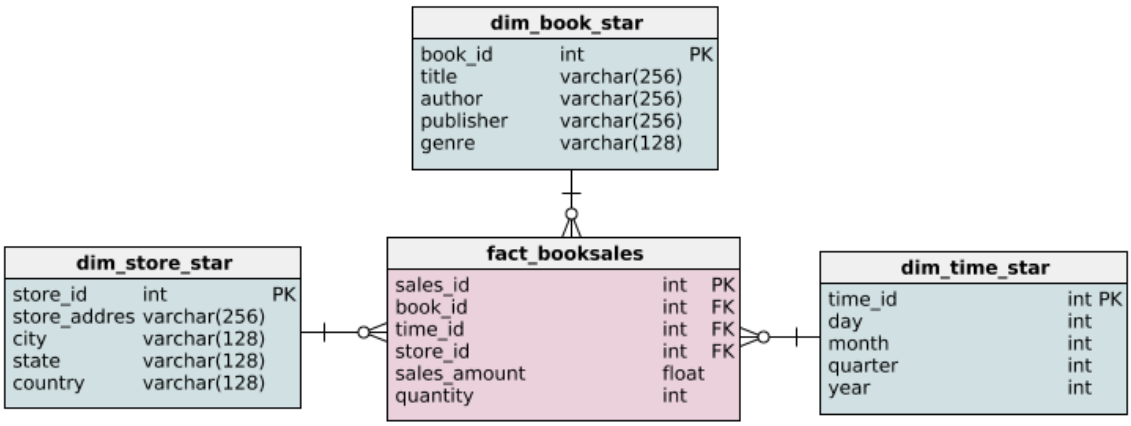
- connect fact tables to dimensions.
- enforce referential integrity.
- create one-to-many relationships when the fact-side FK can repeat while the referenced PK is unique.

The supplied exercise adds three foreign keys to `fact_booksales`: `book_id`, `time_id`, and `store_id`.

```
ALTER TABLE fact_booksales
ADD CONSTRAINT sales_book
FOREIGN KEY (book_id) REFERENCES dim_book_star (book_id);

ALTER TABLE fact_booksales
ADD CONSTRAINT sales_time
FOREIGN KEY (time_id) REFERENCES dim_time_star (time_id);

ALTER TABLE fact_booksales
ADD CONSTRAINT sales_store
FOREIGN KEY (store_id) REFERENCES dim_store_star (store_id);
```



10. Extending a snowflake dimension

The notes create `dim_author` from distinct authors in `dim_book_star`:

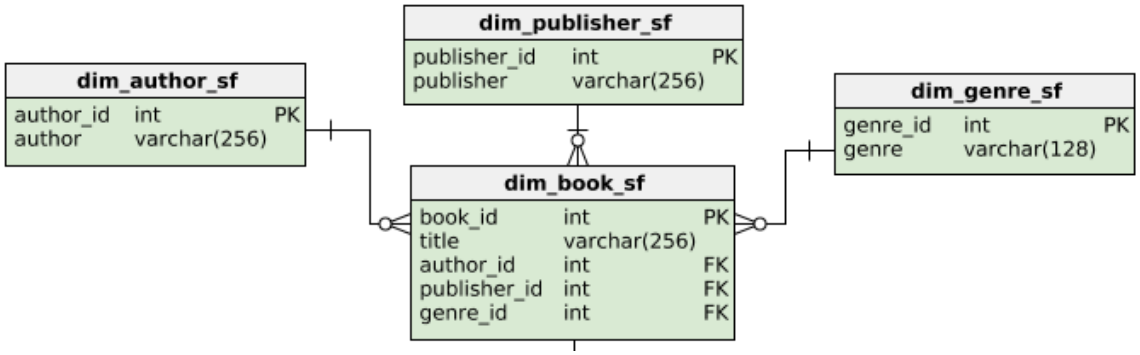
```sql
CREATE TABLE dim_author (
    author VARCHAR(256) NOT NULL
);

INSERT INTO dim_author(author)
SELECT DISTINCT author
FROM dim_book_star;

ALTER TABLE dim_author ADD COLUMN author_id SERIAL PRIMARY KEY;

SELECT * FROM dim_author;
```

This demonstrates how a star dimension can be extended toward a snowflake design by extracting repeated values into their own dimension table.



---

11. Querying star vs snowflake schemas

Star-schema query

Goal: total sales by state for books in the `novel` genre.

```sql
SELECT dim_store_star.state, SUM(fact_booksales.sales_amount)
FROM fact_booksales
JOIN dim_book_star
  ON fact_booksales.book_id = dim_book_star.book_id
JOIN dim_store_star
  ON fact_booksales.store_id = dim_store_star.store_id
WHERE dim_book_star.genre = 'novel'
GROUP BY dim_store_star.state;
```

The star query needs fewer joins because book/store dimensions contain the needed descriptive attributes directly.

Snowflake-schema query

```sql
SELECT dim_state_sf.state, SUM(fact_booksales.sales_amount)
FROM fact_booksales
JOIN dim_book_sf
  ON fact_booksales.book_id = dim_book_sf.book_id
JOIN dim_genre_sf
  ON dim_book_sf.genre_id = dim_genre_sf.genre_id
JOIN dim_store_sf
  ON dim_store_sf.store_id = fact_booksales.store_id
JOIN dim_city_sf
  ON dim_city_sf.city_id = dim_store_sf.city_id
JOIN dim_state_sf
  ON dim_state_sf.state_id = dim_city_sf.state_id
WHERE dim_genre_sf.genre = 'novel'
GROUP BY dim_state_sf.state;
```

The snowflake version requires more joins because normalized dimensions are split into related tables.

Exam rule

> If the required descriptive attribute is already in the dimension → **star query is simpler**.
>
> If the attribute is several relationships away → **snowflake needs more joins**.

---

12. Updating star vs snowflake data

The notes use inconsistent country names as an example. For a star schema, country values are stored directly in `dim_store_star`:

```sql
SELECT *
FROM dim_store_star
WHERE country != 'USA'
  AND country != 'CA';
```

In a snowflake schema, country information can be centralized in `dim_country_sf`, which makes consistency and extension easier.

Extending the snowflake

```
ALTER TABLE dim_country_sf
ADD COLUMN continent_id int NOT NULL DEFAULT(1);

ALTER TABLE dim_country_sf
ADD CONSTRAINT country_continent
FOREIGN KEY (continent_id)
REFERENCES dim_continent_sf(continent_id);

SELECT * FROM dim_country_sf;
```

The exercise emphasizes that a normalized snowflake dimension can be easier to extend while maintaining consistency.

---

13. Database views

13.1 What is a view?

A view is the result set of a stored query. It behaves like a virtual table.

Key points:

- Virtual table.
- Not part of the physical schema as a stored table.
- The **query** is stored, not the result data for a normal/non-materialized view.
- Data comes from underlying tables.
- Can be queried like a regular table.
- Avoids repeatedly typing common complex queries.
- Does not require changing the underlying schema.

Source: chapter 3, p. 2.

13.2 Creating a view

```
CREATE VIEW view_name AS
SELECT col1, col2
FROM table_name
WHERE condition;
```

Source: chapter 3, p. 3.

Example: science-fiction books

```
CREATE VIEW scifi_books AS
SELECT title, author, genre
FROM dim_book_sf
JOIN dim_genre_sf
  ON dim_genre_sf.genre_id = dim_book_sf.genre_id
JOIN dim_author_sf
  ON dim_author_sf.author_id = dim_book_sf.author_id
WHERE dim_genre_sf.genre = 'science fiction';
```

Then:

```
SELECT * FROM scifi_books;
```

The example returns titles, authors, and genre for science-fiction books.

Creating a view (example)

```
CREATE VIEW scifi_books AS
```

```
SELECT title,  author, genre
FROM dim_book_sf
JOIN dim_genre_sf ON dim_genre_sf.genre_id = dim_book_sf.genre_id
JOIN dim_author_sf ON dim_author_sf.author_id = dim_book_sf.author_id
WHERE dim_genre_sf.genre = 'science fiction';
```


13.3 Behind the scenes

A query against the view is effectively equivalent to querying the underlying query result:

```
SELECT * FROM scifi_books;
```

is conceptually equivalent to:

```
SELECT * FROM (
    SELECT title, author, genre
    FROM dim_book_sf
    JOIN dim_genre_sf
      ON dim_genre_sf.genre_id = dim_book_sf.genre_id
    JOIN dim_author_sf
      ON dim_author_sf.author_id = dim_book_sf.author_id
    WHERE dim_genre_sf.genre = 'science fiction'
);
```

Source: chapter 3, p. 7.

13.4 Viewing views in PostgreSQL

All views, including system views:

```
SELECT * FROM information_schema.views;
```

Exclude system views:

```
SELECT *
FROM information_schema.views
WHERE table_schema NOT IN ('pg_catalog', 'information_schema');
```

Source: chapter 3, p. 8.

13.5 Benefits of views

- Little/no storage for a normal view's result.
- Access control.
- Hide sensitive columns.
- Restrict what users can see.
- Mask query complexity.
- Especially useful with highly normalized schemas.

Source: chapter 3, p. 9.

---

14. Managing views

Complex views can use

- Aggregations: `SUM()`, `AVG()`, `COUNT()`, `MIN()`, `MAX()`, `GROUP BY`, etc.
- Joins: `INNER JOIN`, `LEFT JOIN`, `RIGHT JOIN`, `FULL JOIN`.
- Conditions: `WHERE`, `HAVING`, `UNIQUE`, `NOT NULL`, `AND`, `OR`, `>`, `<`, etc.

Source: chapter 3, p. 13.

Granting/revoking view access

```
GRANT privilege(s)
ON object
TO role;

REVOKE privilege(s)
ON object
FROM role;
```

Privileges can include `SELECT`, `INSERT`, `UPDATE`, `DELETE`, etc. Objects include tables, views, schemas, etc. Roles can be database users or groups of users.

Dropping a view

```
DROP VIEW view_name [CASCADE | RESTRICT];
```

- `RESTRICT` is the default and errors if dependent objects exist.
- `CASCADE` drops the view and dependent objects.

Source: chapter 3, p. 19.

Redefining a view

```
CREATE OR REPLACE VIEW view_name AS new_query;
```

Important rules from the slides:

- Existing column names/order/data types must remain compatible.

- New columns can be added at the end.
- If requirements cannot be satisfied, drop and recreate the view.

Source: chapter 3, p. 20.

Notes exercise

The supplied notes correctly answer that `CREATE OR REPLACE VIEW` can be used when the new `label` column is appended at the end.

---

15. Updatable views

Not all views are updatable/insertable.

The slides emphasize that an updatable view is generally simpler, for example a view based on one table and without window/aggregate functions. The course exercise checks `information_schema.views`:

```
SELECT table_name
FROM information_schema.views
WHERE table_schema NOT IN ('pg_catalog','information_schema')
  AND is_updatable = 'YES';
```

In the provided exercise, the answer is `long_reviews`.

Exam caution: prefer modifying base tables rather than relying on writes through complex views.

---

16. Materialized vs non-materialized views

Non-materialized view

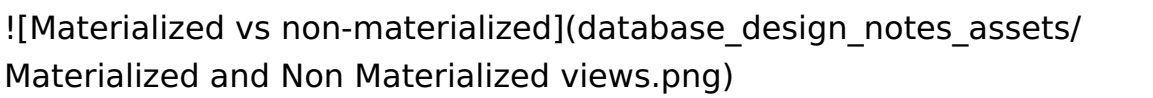
- Stores the query definition.
- Query runs when the view is queried.
- Reflects current underlying data.

- Does not store a separate result set.

Materialized view

- Stores the query **results** physically.
- Querying reads the stored result.
- Faster for expensive/repeated queries when the data can be slightly stale.
- Must be refreshed to reflect new source data.
- Consumes storage.

Source: chapter 3, pp. 24–28.



When to use materialized views

- Long-running queries.
- Underlying results don't change often.
- Data warehouses / OLAP.
- Save computational cost for frequent repeated queries.

Source: chapter 3, p. 27.

PostgreSQL syntax

```
CREATE MATERIALIZED VIEW my_mv AS
SELECT * FROM existing_table;

REFRESH MATERIALIZED VIEW my_mv;
```

Source: chapter 3, p. 28.

Notes exercise

```
CREATE MATERIALIZED VIEW genre_count AS
SELECT genre, COUNT(*)
FROM genres
GROUP BY genre;

INSERT INTO genres
VALUES (50000, 'classical');

REFRESH MATERIALIZED VIEW genre_count;

SELECT * FROM genre_count;
```

The refresh is required because otherwise the materialized view remains a snapshot.

Dependency chains

Materialized views can depend on other materialized views. This creates refresh dependency chains, so refreshing everything simultaneously may be inefficient. DAGs and pipeline schedulers can help manage dependencies.

---

17. Database roles and access control

17.1 Roles

A PostgreSQL role is an entity that contains information about:

- privileges
- whether it can log in
- whether it can create databases
- whether it can write to tables
- authentication information such as passwords

Roles can be assigned to one or more users and are global across a database-cluster installation.

Source: chapter 4, pp. 2–4.

17.2 Create roles

```
CREATE ROLE data_analyst;

CREATE ROLE intern
WITH PASSWORD 'PasswordForIntern'
VALID UNTIL '2020-01-01';

CREATE ROLE admin CREATEDB;

ALTER ROLE admin CREATEROLE;
```

17.3 GRANT and REVOKE

```
GRANT UPDATE ON ratings TO data_analyst;

REVOKE UPDATE ON ratings FROM data_analyst;
```

PostgreSQL privileges shown in the course include:

`SELECT`, `INSERT`, `UPDATE`, `DELETE`, `TRUNCATE`, `REFERENCES`, `TRIGGER`, `CREATE`, `CONNECT`, `TEMPORARY`, `EXECUTE`, `USAGE`.

Source: chapter 4, p. 5.

17.4 User roles and group roles

A role can function as a user and/or group.

```
CREATE ROLE data_analyst;

CREATE ROLE alex
WITH PASSWORD 'PasswordForIntern'
VALID UNTIL '2020-01-01';

GRANT data_analyst TO alex;
REVOKE data_analyst FROM alex;
```

Source: chapter 4, pp. 6–8.

Why group roles matter

Grant permissions to the group once, then add/remove users from that group rather than individually managing every privilege.

---

18. Table partitioning

Why partition?

As tables reach hundreds of GB or TB, queries and updates can slow down. One reason is that indexes may no longer fit comfortably in memory.

Partitioning = split one logical table into smaller physical pieces.

Partitioning belongs to the **physical data model**; the logical model remains conceptually the same.

18.1 Vertical partitioning

Split a table by **columns**.

Example:

```
Original:
[id | name | short_description | price | long_description]

Partition A:
[id | name | short_description | price]

Partition B:
[id | long_description]
```

Use case: move rarely accessed/large columns to a slower medium.

Source: chapter 4, pp. 15–16.

The supplied exercise implements this manually in PostgreSQL:

```sql
CREATE TABLE film_descriptions (
    film_id INT,
    long_description TEXT
);

INSERT INTO film_descriptions
SELECT film_id, long_description FROM film;

ALTER TABLE film DROP COLUMN long_description;

SELECT *
FROM film_descriptions
JOIN film USING(film_id);
```

18.2 Horizontal partitioning

Split a table by **rows**, usually based on a partition key such as date/year.

Example idea:

```
2018 rows → partition 2018
2019 rows → partition 2019
2020 rows → partition 2020
```

It can improve locality and make frequently accessed partitions easier to manage.

Range partitioning example

```sql
CREATE TABLE sales (
    timestamp DATE NOT NULL
)
PARTITION BY RANGE (timestamp);

CREATE TABLE sales_2019_q1 PARTITION OF sales
FOR VALUES FROM ('2019-01-01') TO ('2019-03-31');
```

Source: chapter 4, pp. 17–20.

List partitioning — notes exercise

The exercise partitions `release_year` by explicit values:

```sql
CREATE TABLE film_partitioned (
  film_id INT,
  title TEXT NOT NULL,
  release_year TEXT
)
PARTITION BY LIST (release_year);

CREATE TABLE film_2019
    PARTITION OF film_partitioned FOR VALUES IN ('2019');

CREATE TABLE film_2018
    PARTITION OF film_partitioned FOR VALUES IN ('2018');

CREATE TABLE film_2017
    PARTITION OF film_partitioned FOR VALUES IN ('2017');

INSERT INTO film_partitioned
SELECT film_id, title, release_year FROM film;
```

18.3 Partitioning vs sharding

Partitioning divides a logical table into pieces. **Sharding** extends the idea across multiple machines/nodes.

The course diagram illustrates partitions distributed across machines.

Memory

- **Vertical = columns**.
- **Horizontal = rows**.
- **Sharding = horizontal distribution across machines**.

---

19. Data integration

Definition

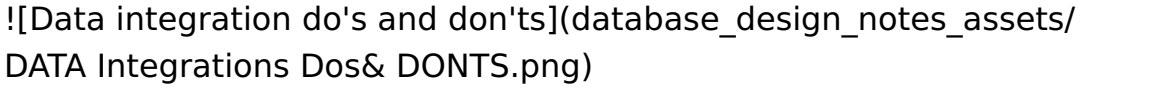
Data integration combines data from different sources, formats, and technologies to provide users with a translated/unified view of the data.

Source: chapter 4, p. 24.

Common business cases

- 360-degree customer view.
- Acquisitions.
- Legacy systems.

The course also emphasizes that real-world integration must account for different source formats, update cadences, transformations, testing, alerts, security, and governance/lineage.



Data integration — exam rules

- Choose tools that are **flexible, reliable, scalable**.
- Automated testing and proactive alerts matter.
- Security matters.
- Sensitive data such as credit-card information may need anonymization.
- Data lineage/governance matters.
- Integration should be **business-driven**: decide what combinations of data are useful before integrating everything.
- Source systems can have different formats and different DBMSs.
- The final analytical view does **not** have to update in real time; update cadence should match business needs.

---

20. Choosing a DBMS — SQL vs NoSQL

DBMS

DBMS = Database Management System.

It creates and maintains databases and provides the interface between users/applications and the database engine/data.

Source: chapter 4, pp. 45–46.

SQL / RDBMS

Relational Database Management System:

- Based on the relational model.
- Uses SQL.
- Strong choice when data is structured and relatively stable.
- Strong choice when consistency is important.

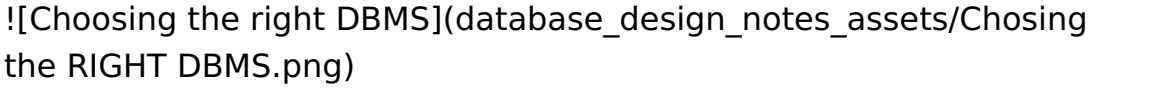
NoSQL

- Less structured.
- Often document-centered rather than table-centered.
- Data does not have to fit rigid rows/columns.
- Useful for rapid growth, unclear schema, and large quantities of data.

Types:

1. Key-value store.
2. Document store.
3. Columnar database.
4. Graph database.

Source: chapter 4, pp. 47–52.



NoSQL types

Type	Core idea	Example use case
Key-value	Unique key → arbitrary value	Shopping cart
Document	Key-value style, but value is structured document	Content management
Columnar	Stores data by columns, scalable	Big-data analytics
Graph	Nodes/edges model relationships	Social networks, recommendations

Exam decision rule

Choose SQL when: structured data + consistency + well-defined relationships are important.

Choose NoSQL when: rapid growth + flexible/unclear schema + huge scale are dominant requirements.

---

21. Assessment screenshot bank — answers

These are the concepts represented by the screenshots supplied with the request.

SQL vs NoSQL

SQL

✅ A banking application where it's extremely important that data integrity is ensured.

Reason: relational SQL systems are a strong fit when consistency/integrity is critical.

NoSQL

The supplied quiz marks these as NoSQL scenarios:

- ✅ A blog that needs new content types such as images, comments, and videos.
- ✅ A social media tool where users grow networks through connections.
- ✅ Data warehousing on big data.
- ✅ An e-commerce website tracking millions of shopping carts.

These map naturally to flexible document, graph, columnar, or key-value workloads.

Conceptual vs logical vs physical data model

Conceptual

- ✅ Entities, attributes, and relationships.
- ✅ Gathers business requirements.

Logical

- ✅ Determines tables and columns.
- ✅ Relational model.

Physical

- ✅ File structure of data storage.

True / False bank

False

- ❌ Automated testing and proactive alerts are not needed.
- ❌ All your data has to be updated in real time in the final view.

True

- ✅ Be careful choosing a hand-coded solution because of maintenance cost.
- ✅ Data integration should be business-driven, e.g. decide what combinations of data will actually be useful.
- ✅ Source data can be in different formats and database management systems.
- ✅ Data in the final view can be updated at different intervals.

Data lake vs data warehouse

Data lake

- ✅ Can include unstructured data.
- ✅ Can hold data whose purpose is not yet determined.

- ✅ Is less organized.

Data warehouse

- ✅ Contains structured data.
- ✅ Holds data where the purpose is known.
- ✅ Is relatively more complicated to change because of upstream/downstream impacts.

Views

Non-materialized views

- ✅ Return up-to-date data.
- ⚠️ Generally useful when the underlying query can be executed at query time; they are not inherently "better" simply because a database is write-intensive.

Both non-materialized and materialized views

- ✅ Can be used in a data warehouse.
- ✅ Can reduce the overhead of repeatedly writing/executing expensive analytical logic, depending on design.

Materialized views

- ✅ Store query results on disk.
- ✅ Consume more storage.

Most important distinction:

```
Non-materialized → store definition/query → execute against source data
Materialized     → store result           → refresh when needed
```

Normalization / partitioning

Normalization

- ✅ Reduces redundancy in tables.

- ✅ Changes the logical data model.

Vertical partitioning

- ✅ Move specific columns to a slower medium.
- ✅ Example: move large third/fourth columns into a separate table.

Horizontal partitioning

- ✅ Sharding is an extension of horizontal partitioning across multiple machines.
- ✅ Example: use a timestamp/date to move rows for a specific period such as Q4 into a partition.

---

22. High-yield SQL syntax sheet

Create table

```
CREATE TABLE table_name (
    id INTEGER PRIMARY KEY,
    name VARCHAR(256) NOT NULL
);
```

Add foreign key

```
ALTER TABLE child_table
ADD CONSTRAINT fk_name
FOREIGN KEY (child_id)
REFERENCES parent_table(parent_id);
```

Add column + FK

```
ALTER TABLE dim_country_sf
ADD COLUMN continent_id INT NOT NULL DEFAULT(1);

ALTER TABLE dim_country_sf
ADD CONSTRAINT country_continent
FOREIGN KEY (continent_id)
REFERENCES dim_continent_sf(continent_id);
```

Create view

```
CREATE VIEW view_name AS
SELECT ...
FROM ...
WHERE ...;
```

Replace view

```
CREATE OR REPLACE VIEW view_name AS
SELECT ...;
```

Drop view

```
DROP VIEW view_name;
DROP VIEW view_name CASCADE;
DROP VIEW view_name RESTRICT;
```

Materialized view

```
CREATE MATERIALIZED VIEW view_name AS
SELECT ...;

REFRESH MATERIALIZED VIEW view_name;
```

Inspect views

```
SELECT *
FROM information_schema.views
WHERE table_schema NOT IN ('pg_catalog', 'information_schema');
```

Inspect updatable views

```
SELECT table_name
FROM information_schema.views
WHERE table_schema NOT IN ('pg_catalog', 'information_schema')
  AND is_updatable = 'YES';
```

Roles

```
CREATE ROLE analyst;
CREATE ROLE user1 WITH LOGIN;
CREATE ROLE admin WITH CREATEDB CREATEROLE;
```

Privileges

```
GRANT SELECT ON table_name TO analyst;
GRANT UPDATE, INSERT ON view_name TO analyst;
REVOKE UPDATE, INSERT ON view_name FROM PUBLIC;
```

Group membership

```
GRANT analyst TO user1;
REVOKE analyst FROM user1;
```

Vertical partitioning

```
CREATE TABLE descriptions (
    id INT,
    description TEXT
);

INSERT INTO descriptions
SELECT id, description FROM original;

ALTER TABLE original DROP COLUMN description;
```

Horizontal list partitioning

```
CREATE TABLE parent (
    id INT,
    year TEXT
)
PARTITION BY LIST (year);

CREATE TABLE parent_2019
PARTITION OF parent FOR VALUES IN ('2019');
```

23. Exam traps and corrections

1. **OLTP is not analytics.** OLTP = transactions; OLAP = analysis.
2. **A view is not normally a stored copy of its result.** A materialized view is.
3. **Materialized views can become stale.** Refresh them.
4. **Normalization reduces redundancy but can increase joins/CPU.**
5. **1NF = atomic values.**
6. **2NF = no partial dependency on a composite key.**
7. **3NF = no transitive dependency.**
8. **Star = denormalized dimensions; snowflake = normalized dimensions.**
9. **Vertical partitioning = columns; horizontal partitioning = rows.**
10. **Partitioning is physical design; normalization changes logical design.**
11. **Sharding distributes data across machines.**
12. **SQL is not automatically "better" than NoSQL.** Choose based on requirements.
13. **Data integration is business-driven, not "integrate everything".**
14. **Final analytical views do not necessarily need real-time updates.**
15. **Write access should not be the default.** Use roles and least privilege.
16. `CREATE OR REPLACE VIEW` **has column compatibility rules.** New columns can be appended at the end under the stated conditions.
17. **The supplied notes contain a likely typo:** the week dimension is shown as `CREATE TABLE route`; use a week-specific table name in a real implementation.

---

24. One-page memory map

```
DATABASE DESIGN
│
├── Processing
│   ├── OLTP → transactions, current, writes
│   └── OLAP → analytics, historical, reads
│
├── Storage
│   ├── Traditional DB → structured / real-time
│   ├── Warehouse → structured / OLAP
│   └── Lake → all structures / flexible / schema-on-read
│
├── Modeling
│   ├── Conceptual → entities + relationships + attributes
│   ├── Logical → tables + columns + relationships
│   └── Physical → storage + indexes + partitions
│
├── Dimensional modeling
│   ├── Fact → metrics
│   ├── Dimension → descriptions
│   ├── Star → denormalized dimensions
│   └── Snowflake → normalized dimensions
│
├── Normalization
│   ├── 1NF → atomic values
│   ├── 2NF → no partial dependency
│   └── 3NF → no transitive dependency
│
├── Views
│   ├── Normal → query definition
│   ├── Materialized → stored results
│   ├── CREATE VIEW
│   ├── CREATE OR REPLACE VIEW
│   └── REFRESH MATERIALIZED VIEW
│
├── Security
│   ├── Roles
│   ├── GRANT
│   └── REVOKE
│
├── Performance / scale
│   ├── Vertical partition → columns
│   ├── Horizontal partition → rows
│   └── Sharding → across machines
│
└── DBMS
    ├── SQL/RDBMS → structure + consistency
    └── NoSQL → flexibility + scale + unclear schema
```

25. Visual reference gallery

The following visual references were extracted from the supplied chapter PDFs or included directly with `notes.md`.

Data modeling / storage

Data modeling

Process of creating a *data model* for the data to be stored

1. Conceptual data model: describes entities, relationships, and attributes

- *Tools:* data structure diagrams, e.g., entity-relational diagrams and UML diagrams

2. Logical data model: defines tables, columns, relationships

- *Tools:* database models and schemas, e.g., relational model and star schema

3. Physical data model: describes physical storage

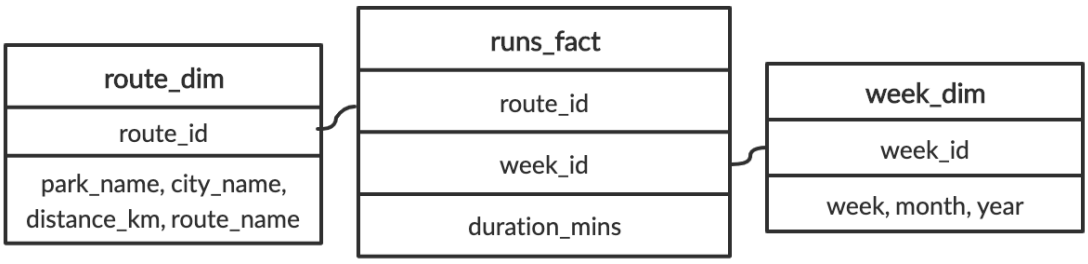
- *Tools*: partitions, CPUs, indexes, backup systems and tablespaces

[1] https://en.wikipedia.org/wiki/Data_model

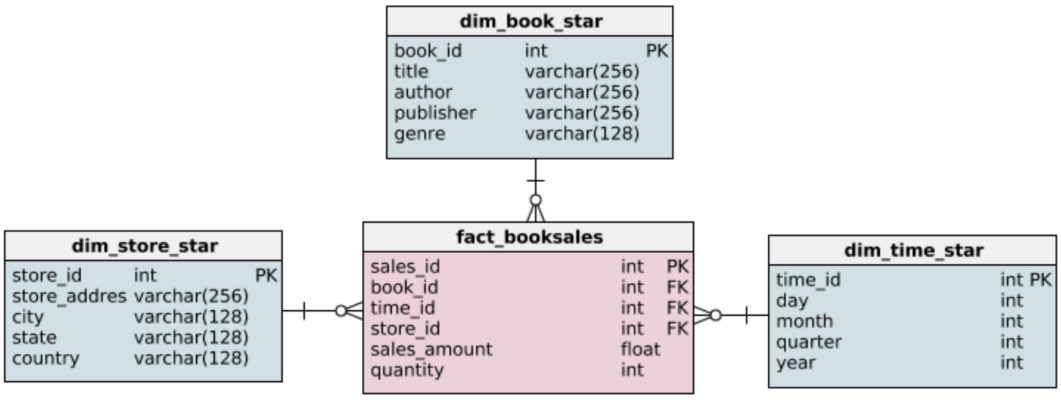
datacamp DATABASE DESIGN

![Data warehouse vs data lake](database_design_notes_assets/DW VS DL.png)

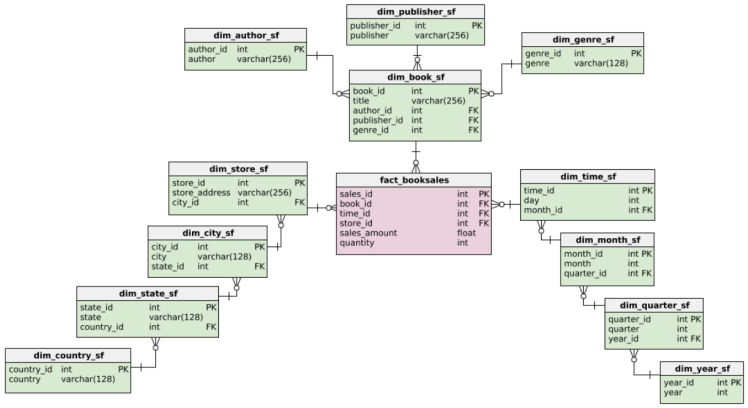
Dimensional modeling

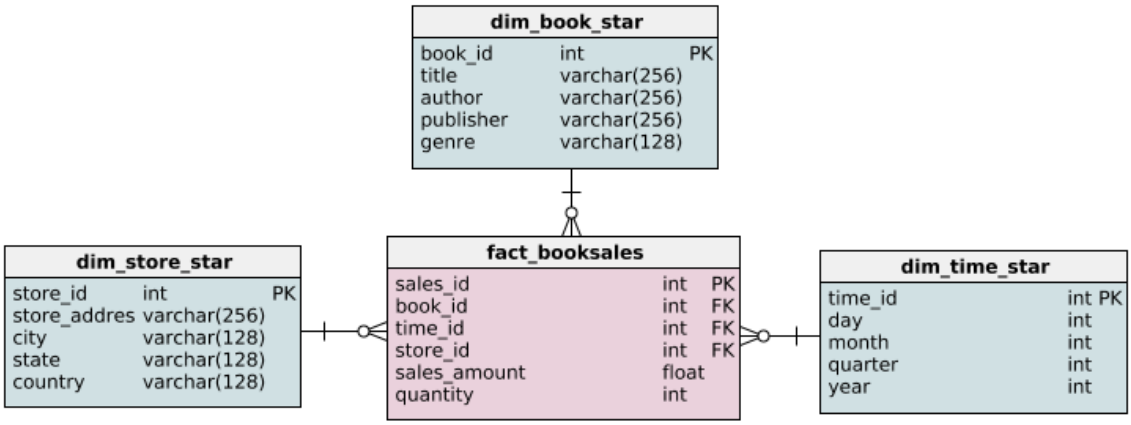


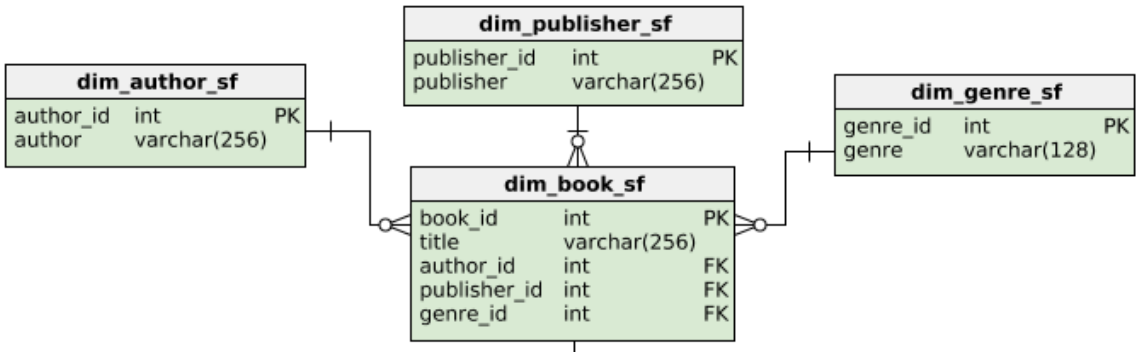
Star schema example



Snowflake schema (an extension)







Normalization

In 1NF form

Student_id	Student_Email
235	jim@gmail.com
455	kelly@yahoo.com
767	amy@hotmail.com

Student_id	Completed
235	Introduction to Python
235	Intermediate Python
455	Cleaning Data in R
767	Machine Learning Toolbox
767	Deep Learning in Python

In 2NF form

Student_id (PK)	Course_id (PK)	Percent_Completed
235	2001	.55
455	2345	.10
767	6584	1.00

Course_id (PK)	Instructor_id	Instructor
2001	560	Nick Carchedi
2345	658	Ginger Grant
6584	999	Chester Ismay

In 3NF

Course_id (PK)	Instructor	Tech
2001	Nick Carchedi	Python
2345	Ginger Grant	SQL
6584	Chester Ismay	R

Instructor_id	Instructor
560	Nick Carchedi
658	Ginger Grant
999	Chester Ismay

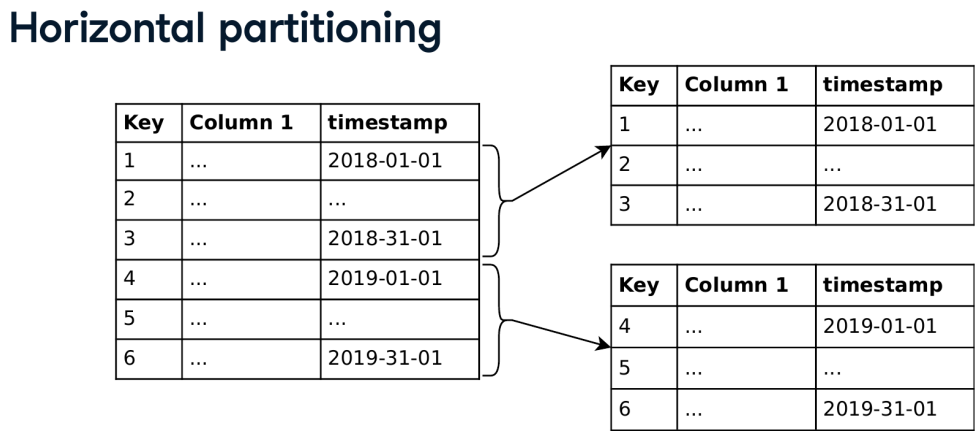
Views

Creating a view (example)

```
CREATE VIEW scifi_books AS
```

```
SELECT title,  author, genre
FROM dim_book_sf
JOIN dim_genre_sf ON dim_genre_sf.genre_id = dim_book_sf.genre_id
JOIN dim_author_sf ON dim_author_sf.author_id = dim_book_sf.author_id
WHERE dim_genre_sf.genre = 'science fiction';
```

Partitioning / integration / DBMS



datacamp DATABASE DESIGN

![Data integration](database_design_notes_assets/DATA Integrations Dos& DONTS.png)

![Choosing a DBMS](database_design_notes_assets/Chosing the RIGHT DBMS.png)

---

26. Complete `notes.md` source text

> This appendix preserves the text from the supplied `notes.md` so that none of the original exercise wording/code is lost. The cleaned notes above are the study-oriented interpretation.

```
Deciding fact and dimension tables
Imagine that you love running and data. It's only natural that you begin collecting data on your weekly running routine. You're most concerned with tracking how long you are running each week. You also record the route and the distances of your runs. You gather this data and put it into one table called Runs with the following schema:

runs
duration_mins - float
week - int
month - varchar(160)
year - int
park_name - varchar(160)
city_name - varchar(160)
distance_km - float
route_name - varchar(160)
After learning about dimensional modeling, you decide to restructure the schema for the database. Runs has been pre-loaded for you.

Instructions 1/2
Question
Out of these possible answers, what would be the best way to organize the fact table and dimensional tables?

Possible answers

ANSWER.1
1.A fact table holding duration_mins, distance_km and foreign keys to dimension tables holding route details and week details, respectively.

2.A fact table holding week,month, year and foreign keys to dimension tables holding route details and duration details, respectively.

3.A fact table holding route_name,park_name,city_name, and foreign keys to dimension tables holding week details and duration details, respectively.

--Create a dimension table called route that will hold the route information.
--Create a dimension table called week that will hold the week information.
-- Create a route dimension table
CREATE TABLE route(
    route_id INTEGER PRIMARY KEY,
    route_name VARCHAR(160) NOT NULL,
    park_name VARCHAR(160) NOT NULL,
    distance_km INTEGER NOT NULL,
    city_name VARCHAR(160) NOT NULL
);
-- Create a week dimension table
CREATE TABLE route(
    week_id INTEGER PRIMARY KEY,
    week INTEGER NOT NULL,
    month VARCHAR(160) NOT NULL,
    year INTEGER NOT NULL
);

============
Querying the dimensional model
Here it is! The schema reorganized using the dimensional model:
![alt text](database_design_notes_assets/run_dim.png)
Let's try to run a query based on this schema. How about we try to find the number of minutes we ran in July, 2019? We'll break this up in two steps. First, we'll get the total number of minutes recorded in the database. Second, we'll narrow down that query to week_id's from July, 2019.

--Calculate the sum of the duration_mins column.
```

```
SELECT
    -- Select the sum of the duration of all runs
    SUM(duration_mins)
FROM
    runs_fact;

--Join week_dim and runs_fact.
--Get all the week_id's from July, 2019.
SELECT
    -- Get the total duration of all runs
    SUM(duration_mins)
FROM
    runs_fact
-- Get all the week_id's that are from July, 2019
INNER JOIN week_dim ON  runs_fact.week_id =week_dim.week_id
WHERE month = 'July' and year = '2019';
```

==========

Adding foreign keys

Foreign key references are essential to both the snowflake and star schema. When creating either of these schemas, correctly setting up the foreign keys is vital because they connect dimensions to the fact table. They also enforce a one-to-many relationship, because unless otherwise specified, a foreign key can appear more than once in a table and primary key can appear only once.

The fact_booksales table has three foreign keys: book_id, time_id, and store_id. In this exercise, the four tables that make up the star schema below have been loaded. However, the foreign keys still need to be added.

Instructions

In the constraint called sales_book, set book_id as a foreign key.
In the constraint called sales_time, set time_id as a foreign key.
In the constraint called sales_store, set store_id as a foreign key.

```
-- Add the book_id foreign key
ALTER TABLE fact_booksales ADD CONSTRAINT sales_book
    FOREIGN KEY (book_id) REFERENCES dim_book_star (book_id);

-- Add the time_id foreign key
ALTER TABLE fact_booksales ADD CONSTRAINT sales_time
    FOREIGN KEY (time_id) REFERENCES dim_time_star (time_id);

-- Add the store_id foreign key
ALTER TABLE fact_booksales ADD CONSTRAINT sales_store
    FOREIGN KEY (store_id) REFERENCES dim_store_star (store_id)
```

==========

Extending the book dimension

In the video, we saw how the book dimension differed between the star and snowflake schema. The star schema's dimension table for books, dim_book_star, has been loaded and below is the snowflake schema of the book dimension.

Extending the book dimension

In the video, we saw how the book dimension differed between the star and snowflake schema. The star schema's dimension table for books, dim_book_star, has been loaded and below is the snowflake schema of the book dimension.

In this exercise, you are going to extend the star schema to meet part of the snowflake schema's criteria. Specifically, you will create dim_author from the data provided in dim_book_star.

In this exercise, you are going to extend the star schema to meet part of the snowflake schema's criteria. Specifically, you will create dim_author from the data provided in dim_book_star.

```
--Instructions 1/2
--Create dim_author with a column for author.
```

```
--Insert all the distinct authors from dim_book_star into dim_author.
-- Create dim_author with an author column
-- Create dim_author with an author column
CREATE TABLE dim_author (
    author VARCHAR(256)  NOT NULL
);

-- Insert distinct authors
INSERT INTO dim_author(author)
SELECT DISTINCT author
FROM dim_book_star;

--Alter dim_author to have a primary key called author_id.
--Output all the columns of dim_author.
-- Add a primary key
ALTER TABLE dim_author ADD COLUMN author_id SERIAL PRIMARY KEY;

-- Output the new table
SELECT * FROM dim_author;
======================
Querying the star schema
The novel genre hasn't been selling as well as your company predicted. To help remedy this, you've been tasked to run some analytics on the novel genre to find which areas the Sales team should target. To begin, you want to look at the total amount of sales made in each state from books in the novel genre.
Luckily, you've just finished setting up a data warehouse with the following star schema:
The tables from this schema have been loaded. Note that you should not use aliases in FROM and JOIN statements.

Instructions
![alt text](database_design_notes_assets/book-star.png)
Select state from the appropriate table and the total sales_amount.
Complete the JOIN on book_id.
Complete the JOIN to connect the dim_store_star table
Conditionally select for books with the genre novel.
Group the results by state.
==========================
-- Output each state and their total sales_amount
SELECT dim_store_star.state, SUM(fact_booksales.sales_amount)
FROM fact_booksales
    -- Join to get book information
    JOIN dim_book_star ON fact_booksales.book_id = dim_book_star.book_id
    -- Join to get store information
    JOIN dim_store_star ON fact_booksales.store_id = dim_store_star.store_id
-- Get all books with in the novel genre
WHERE
    dim_book_star.genre = 'novel'
-- Group results by state
GROUP BY
    dim_store_star.state;
====================

Querying the snowflake schema
Imagine that you didn't have the data warehouse set up. Instead, you'll have to run this query on the company's operational database, which means you'll have to rewrite the previous query with the following snowflake schema:
![alt text](database_design_notes_assets/book-snowflake-copy.png)
The tables in this schema have been loaded. Remember, our goal is to find the amount of money made from the novel genre in each state.

Instructions

Select state from the appropriate table and the total sales_amount.
Complete the two JOINS to get the genre_id's.
```

```
Complete the three JOINS to get the state_id's.
Conditionally select for books with the genre novel.
Group the results by state.
-- Output each state and their total sales_amount
SELECT dim_state_sf.state , SUM(fact_booksales.sales_amount)
FROM fact_booksales
    -- Joins for genre
    JOIN dim_book_sf on fact_booksales.book_id  = dim_book_sf.book_id
    JOIN dim_genre_sf on dim_book_sf.genre_id   =dim_genre_sf.genre_id
    -- Joins for state
    JOIN  dim_store_sf on dim_store_sf.store_id = fact_booksales.store_id
    JOIN  dim_city_sf on dim_city_sf.city_id = dim_store_sf.city_id
    JOIN  dim_state_sf on dim_state_sf.state_id = dim_city_sf.state_id
-- Get all books with in the novel genre and group the results by state
WHERE
    dim_genre_sf.genre = 'novel'
GROUP BY
    dim_state_sf.state ;
====================
Updating countries
Going through the company data, you notice there are some inconsistencies in the store
addresses. These probably occurred during data entry, where people fill in fields using
different naming conventions. This can be especially seen in the country field, and you decide
that countries should be represented by their abbreviations. The only countries in the
database are Canada and the United States, which should be represented as USA and CA.

In this exercise, you will compare the records that need to be updated in order to do this
task on the star and snowflake schema. dim_store_star and dim_country_sf have been loaded.

Instructions

Output all the records that need to be updated in the star schema so that countries are
represented by their abbreviations.
-- Output records that need to be updated in the star schema
SELECT * FROM dim_store_star
WHERE country != 'USA' AND country !='CA';
=========
Exercise
Extending the snowflake schema
The company is thinking about extending their business beyond bookstores in Canada and the US.
Particularly, they want to expand to a new continent. In preparation, you decide a continent
field is needed when storing the addresses of stores.

Luckily, you have a snowflake schema in this scenario. As we discussed in the video, the
snowflake schema is typically faster to extend while ensuring data consistency. Along with
dim_country_sf, a table called dim_continent_sf has been loaded. It contains the only
continent currently needed, North America, and a primary key. In this exercise, you'll need to
extend dim_country_sf to reference dim_continent_sf.

Instructions

--Add a continent_id column to dim_country_sf with a default value of 1. Note thatNOT NULL
DEFAULT(1) constrains a value from being null and defaults its value to 1.
--Make that new column a foreign key reference to dim_continent_sf's continent_id.
-- Add a continent_id column with default value of 1
ALTER TABLE dim_country_sf
ADD COLUMN continent_id int NOT NULL DEFAULT(1);

-- Add the foreign key constraint
ALTER TABLE dim_country_sf ADD CONSTRAINT country_continent
   FOREIGN KEY (continent_id) REFERENCES dim_continent_sf(continent_id);

-- Output updated table
SELECT * FROM dim_country_sf;
```

```
================Converting to 1NF===========
In the next three exercises, you'll be working through different tables belonging to a car
rental company. Your job is to explore different schemas and gradually increase the
normalization of these schemas through the different normal forms. At this stage, we're not
worried about relocating the data, but rearranging the tables.

A table called customers has been loaded, which holds information about customers and the cars
they have rented.
cars_rented holds one or more car_ids and invoice_id holds multiple values. Create a new table
to hold individual car_ids and invoice_ids of the customer_ids who've rented those cars.
Drop two columns from customers table to satisfy 1NF

-- Create a new table to hold the cars rented by customers
CREATE TABLE cust_rentals (
  customer_id INT NOT NULL,
  car_id VARCHAR(128) NULL,
  invoice_id VARCHAR(128) NULL
);

-- Drop two columns from customers table to satisfy 1NF
ALTER TABLE customers
DROP COLUMN cars_rented,
DROP COLUMN Invoice_id;
==================Converting to 2NF==================
Let's try normalizing a bit more. In the last exercise, you created a table holding
customer_ids and car_ids. This has been expanded upon and the resulting table,
customer_rentals, has been loaded for you. Since you've got 1NF down, it's time for 2NF.

Create a new table for the non-key columns that were conflicting with 2NF criteria.
Drop those non-key columns from customer_rentals.
-- Create a new table to satisfy 2NF
CREATE TABLE cars (
  car_id VARCHAR(256) NULL,
  model VARCHAR(128),
  manufacturer VARCHAR(128),
  type_car VARCHAR(128),
  condition VARCHAR(128),
  color VARCHAR(128)
);

-- Insert data into the new table
INSERT INTO cars
SELECT DISTINCT
  car_id,
  model,
  manufacturer,
  type_car,
  condition,
  color
FROM customer_rentals;

-- Drop columns in customer_rentals to satisfy 2NF
ALTER TABLE customer_rentals
DROP COLUMN model,
DROP COLUMN manufacturer,
DROP COLUMN type_car,
DROP COLUMN condition,
DROP COLUMN color;
============Converting to 3NF===========
Last, but not least, we are at 3NF. In the last exercise, you created a table holding car_idss
and car attributes. This has been expanded upon. For example, car_id is now a primary key. The
resulting table, rental_cars, has been loaded for you.
Instructions
```

```
Create a new table for the non-key columns that were conflicting with 3NF criteria.
Drop those non-key columns from rental_cars.
-- Create a new table to satisfy 3NF

-- Create a new table to satisfy 3NF
CREATE TABLE car_model(
  model VARCHAR(128),
  manufacturer VARCHAR(128),
  type_car VARCHAR(128)
);

-- Drop columns in rental_cars to satisfy 3NF
ALTER TABLE rental_cars
DROP COLUMN manufacturer,
DROP COLUMN type_car;
==============VIEWS======================
Exercise
Viewing views
Because views are very useful, it's common to end up with many of them in your database. It's
important to keep track of them so that database users know what is available to them.

The goal of this exercise is to get familiar with viewing views within a database and
interpreting their purpose. This is a skill needed when writing database documentation or
organizing views.

Instructions
Query the information schema to get views.
Exclude system views in the results.
-- Get all non-systems views
SELECT * FROM information_schema.views
WHERE table_schema NOT IN ('pg_catalog', 'information_schema');
=========
Creating and querying a view
Create a view called high_scores that holds reviews with scores above a 9.
-- Create a view for reviews with a score above 9
CREATE VIEW high_scores AS
SELECT * FROM reviews
WHERE score > 9;

--Count the number of records in high_scores that are self-released in the label --field of
the labels table.
-- Create a view for reviews with a score above 9
CREATE VIEW high_scores AS
SELECT * FROM REVIEWS
WHERE score > 9;

-- Count the number of self-released works in high_scores
SELECT COUNT(*) FROM labels
INNER JOIN high_scores ON high_scores.reviewid = labels.reviewid
WHERE labels.label = 'self-released';
-------------------------
Exercise
Creating a view from other views
Views can be created from queries that include other views. This is useful when you have a
complex schema, potentially due to normalization, because it helps reduce the JOINS needed.
The biggest concern is keeping track of dependencies, specifically how any modifying or
dropping of a view may affect other views.

In the next few exercises, we'll continue using the Pitchfork reviews data. There are two
views of interest in this exercise. top_15_2017 holds the top 15 highest scored reviews
published in 2017 with columns reviewid,title, and score. artist_title returns a list of all
reviewed titles and their respective artists with columns reviewid, title, and artist. From
these views, we want to create a new view that gets the highest scoring artists of 2017.
```

```
INSTRUCTIONS
Create a view called top_artists_2017 with artist from artist_title.
To only return the highest scoring artists of 2017, join the views top_15_2017 and
artist_title on reviewid.
Output top_artists_2017
-- Create a view with the top artists in 2017
CREATE VIEW top_artists_2017 AS
-- with only one column holding the artist field
SELECT r.artist FROM artist_title as r
INNER JOIN top_15_2017 as t
ON r.reviewid = t.reviewid;

-- Output the new view
SELECT * FROM top_artists_2017;
==========
Granting and revoking access
Access control is a key aspect of database management. Not all database users have the same
needs and goals, from analysts, clerks, data scientists, to data engineers. As a general rule
of thumb, write access should never be the default and only be given when necessary.

In the case of our Pitchfork reviews, we don't want all database users to be able to write
into the long_reviews view. Instead, the editor should be the only user able to edit this
view.
Instructions

Revoke all database users' update and insert privileges on the long_reviews view.
Grant the editor user update and insert privileges on the long_reviews view.
-- Revoke everyone's update and insert privileges
REVOKE UPDATE, INSERT ON long_reviews FROM PUBLIC;

-- Grant the editor update and insert privileges
GRANT UPDATE, INSERT ON long_reviews TO editor;
===Exercise====
Updatable views
In a previous exercise, we've used the information_schema.views to get all the views in a
database. If you take a closer look at this table, you will notice a column that indicates
whether the view is updatable.

Which views are updatable?
SELECT table_name From information_schema.views
WHERE table_schema NOT IN ('pg_catalog','information_schema') AND Is_updatable =  'YES'
Check table name with running this query and see which views are updatable.
Answer: long_reviews

==========
Redefining a view
Unlike inserting and updating, redefining a view doesn't mean modifying the actual data a view
holds. Rather, it means modifying the underlying query that makes the view. In the last video,
we learned of two ways to redefine a view: (1) CREATE OR REPLACE and (2) DROP then CREATE.
CREATE OR REPLACE can only be used under certain conditions.

The artist_title view needs to be appended to include a column for the label field from the
labels table.

Question
Can the CREATE OR REPLACE statement be used to redefine the artist_title view?

Possible answers
Yes, as long as the label column comes at the end.
No, because the new query requires a JOIN with the labels table.
No, because a new column that did not exist previously is being added to the view.
Yes, as long as the label column has the same data type as the other columns in artist_title

ANSWER:Yes, as long as the label column comes at the end.
```

```
INSTRUCTIONS
Use CREATE OR REPLACE to redefine the artist_title view.
Respecting artist_title's original columns of reviewid, title, and artist, add a label column
from the labels table.
Join the labels table using the reviewid field

-- Redefine the artist_title view to have a label column
CREATE OR REPLACE VIEW artist_title AS
SELECT reviews.reviewid, reviews.title, artists.artist, labels.label
FROM reviews
INNER JOIN artists
ON artists.reviewid = reviews.reviewid
INNER JOIN labels
ON labels.reviewid = reviews.reviewid;

SELECT * FROM artist_title;
==========QUIZ=========
![alt text](<Materialized and Non Materialized views.png>)
===========================

Exercise
Creating and refreshing a materialized view
The syntax for creating materialized and non-materialized views are quite similar because they
are both defined by a query. One key difference is that we can refresh materialized views,
while no such concept exists for non-materialized views. It's important to know how to refresh
a materialized view, otherwise the view will remain a snapshot of the time the view was
created.

In this exercise, you will create a materialized view from the table genres. A new record will
then be inserted into genres. To make sure the view has the latest data, it will have to be
refreshed.
INSTRUCTIONS
Create a materialized view called genre_count that holds the number of reviews for each genre.
Refresh genre_count so that the view is up-to-date.
===============
Creating and refreshing a materialized view
The syntax for creating materialized and non-materialized views are quite similar because they
are both defined by a query. One key difference is that we can refresh materialized views,
while no such concept exists for non-materialized views. It's important to know how to refresh
a materialized view, otherwise the view will remain a snapshot of the time the view was
created.

In this exercise, you will create a materialized view from the table genres. A new record will
then be inserted into genres. To make sure the view has the latest data, it will have to be
refreshed.
----------
INSTRUCTIONS
Create a materialized view called genre_count that holds the number of reviews for each genre.
Refresh genre_count so that the view is up-to-date.
-- Create a materialized view called genre_count
CREATE MATERIALIZED VIEW genre_count AS
SELECT genre, COUNT(*)
FROM genres
GROUP BY genre;

INSERT INTO genres
VALUES (50000, 'classical');

-- Refresh genre_count
REFRESH MATERIALIZED VIEW genre_count;

SELECT * FROM genre_count;
=================
```

```
Exercise
Create a role
A database role is an entity that contains information that define the role's privileges and
interact with the client authentication system. Roles allow you to give different people (and
often groups of people) that interact with your data different levels of access.

Imagine you founded a startup. You are about to hire a group of data scientists. You also
hired someone named Marta who needs to be able to login to your database. You're also about to
hire a database administrator. In this exercise, you will create these roles.
INSTRUCTIONS

Create a role called data_scientist.

Create a role called marta that has one attribute: the ability to login (LOGIN).

Create a role called admin with the ability to create databases (CREATEDB) and to create roles
(CREATEROLE).
CREATE ROLE data_scientist;

-- Create a role for Marta
CREATE ROLE marta WITH LOGIN

-- Create an admin role
CREATE ROLE admin WITH CREATEDB CREATEROLE;
=========
Exercise
GRANT privileges and ALTER attributes
Once roles are created, you grant them specific access control privileges on objects, like
tables and views. Common privileges being SELECT, INSERT, UPDATE, etc.

Imagine you're a cofounder of that startup and you want all of your data scientists to be able
to update and insert data in the long_reviews view. In this exercise, you will enable those
soon-to-be-hired data scientists by granting their role (data_scientist) those privileges.
Also, you'll give Marta's role a password.
-----------
Instructions

Grant the data_scientist role update and insert privileges on the long_reviews view.
Alter Marta's role to give her the provided password.
-- Grant data_scientist update and insert privileges
GRANT UPDATE, INSERT  ON long_reviews TO data_scientist;

-- Give Marta's role a password
ALTER ROLE marta WITH PASSWORD 's3cur3p@ssw0rd';

----------
Exercise
Add a user role to a group role
There are two types of roles: user roles and group roles. By assigning a user role to a group
role, a database administrator can add complicated levels of access to their databases with
one simple command.

For your startup, your search for data scientist hires is taking longer than expected.
Fortunately, it turns out that Marta, your recent hire, has previous data science experience
and she's willing to chip in the interim. In this exercise, you'll add Marta's user role to
the data scientist group role. You'll then remove her after you complete your hiring process.

Instructions
Add Marta's user role to the data scientist group role.
Celebrate! You hired multiple data scientists.
Remove Marta's user role from the data scientist group role.

-- Add Marta to the data scientist group
GRANT data_scientist TO marta;
```

```
-- Celebrate! You hired data scientists.

-- Remove Marta from the data scientist group
REVOKE data_scientist FROM marta;
```



```
----------------------------
Exercise
Creating vertical partitions
In the video, you learned about vertical partitioning and saw an example.

For vertical partitioning, there is no specific syntax in PostgreSQL. You have to create a new table with particular columns and copy the data there. Afterward, you can drop the columns you want in the separate partition. If you need to access the full table, you can do so by using a JOIN clause.

In this exercise and the next one, you'll be working with the example database called pagila. It's a database that is often used to showcase PostgreSQL features. The database contains several tables. We'll be working with the film table. In this exercise, we'll use the following columns:

film_id: the unique identifier of the film
long_description: a lengthy description of the film
-------
INSTRUCTIONS
Create a new table film_descriptions containing 2 fields: film_id, which is of type INT, and long_description, which is of type TEXT.
Occupy the new table with values from the film table.
-- Create a new table called film_descriptions
CREATE TABLE film_descriptions (
    film_id INT,
    long_description TEXT
);

-- Copy the descriptions from the film table
INSERT INTO film_descriptions
SELECT film_id, long_description FROM film;

--Drop the field long_description from the film table.
--Join the two resulting tables to view the original table.
-- Create a new table called film_descriptions
CREATE TABLE film_descriptions (
    film_id INT,
    long_description TEXT
);

-- Copy the descriptions from the film table
INSERT INTO film_descriptions
SELECT film_id, long_description FROM film;

-- Drop the descriptions from the original table
ALTER TABLE film DROP COLUMN long_description;

-- Join to view the original table
SELECT * FROM film_descriptions
JOIN film USING(film_id);
----------------
Exercise
Creating horizontal partitions
In the video, you also learned about horizontal partitioning.

The example of horizontal partitioning showed the syntax necessary to create horizontal partitions in PostgreSQL. If you need a reminder, you can have a look at the slides.
```

```
In this exercise, however, you'll be using a list partition instead of a range partition. For
list partitions, you form partitions by checking whether the partition key is in a list of
values or not.

To do this, we partition by LIST instead of RANGE. When creating the partitions, you should
check if the values are IN a list of values.

We'll be using the following columns in this exercise:

film_id: the unique identifier of the film
title: the title of the film
release_year: the year it's released
--INSTRUCTIONS
--Create the table film_partitioned, partitioned on the field release_year.
-- Create a new table called film_partitioned
CREATE TABLE film_partitioned (
  film_id INT,
  title TEXT NOT NULL,
  release_year TEXT
)
PARTITION BY RANGE (release_year);
---
--Create three partitions: one for each release year: 2017, 2018, and 2019. Call the partition
for 2019 film_2019, etc.
-- Create a new table called film_partitioned
CREATE TABLE film_partitioned (
  film_id INT,
  title TEXT NOT NULL,
  release_year TEXT
)
PARTITION BY LIST (release_year);

-- Create the partitions for 2019, 2018, and 2017
CREATE TABLE film_2019
    PARTITION OF film_partitioned FOR VALUES IN ('2019');

CREATE TABLE film_2018
    PARTITION OF film_partitioned FOR VALUES IN ('2018');

CREATE TABLE film_2017
    PARTITION OF film_partitioned FOR VALUES IN ('2017');

--Occupy the new table, film_partitioned, with the three fields required from the film table.
-- Insert the data into film_partitioned
INSERT INTO film_partitioned
SELECT film_id, title,release_year FROM film;

-- View film_partitioned
SELECT * FROM film_partitioned;
![alt text](<DATA Integrations Dos& DONTS.png>)
![alt text](<Chosing the RIGHT DBMS.png>)
![alt text](<DW VS DL.png>)
```

Source files analyzed

- `notes.md` — primary exercise/answer notes; 577 rendered source lines.